

Contents

1	Contract State Machine Documentation	1
1.1	Overview	1
1.2	JSON Schema Definitions	1
1.3	Contract State Schema	1
1.4	Contract Transition Schema	2
1.5	Merkle Proof Schema	3
1.6	WIT Interface Definitions	3
1.7	Hooksmith CLI Interface	3
1.8	Hook Builder Interface	5
1.9	Validation Interface	9
1.10	Lefthook Generator Interface	18
1.11	Integration with WIT & JSON Schema	25
1.12	Benefits	25

1 Contract State Machine Documentation

This document provides a comprehensive overview of Hooksmith’s contract validation state machine, including JSON schemas and WIT interfaces.

1.1 Overview

Hooksmith implements a schema-driven state machine for contract validation that provides:

- **State Machine:** Enforces valid state transitions (UNTRACKED → UNVALIDATED → VALIDATED → LOCKED)
- **Merkle Chain:** Cryptographic proof of integrity across hierarchical scopes
- **Git Notes Integration:** Tamper-proof audit trails with full validation history
- **CI Enforcement:** Automated validation and security auditing in GitHub Actions

1.2 JSON Schema Definitions

The following schemas define the structure and validation rules for the contract state machine:

1.3 Contract State Schema

Defines the structure for contract validation states.

Property	Type	Required	Description
contract	string		Type of contract (blob, tree, commit, tag, etc.)
file	string		Relative path to the file from repository root
hash	string		SHA-256 hash of the validated content
metadata	object		Additional contract-specific metadata
parent_hash	string		Hash of parent scope (for Merkle tree)
parent_scope	string		Parent scope identifier (for Merkle tree)
state	object		Current state of the contract
timestamp	string		ISO 8601 timestamp of validation
validated_by	string		Tool and version that performed validation

1.4 Contract Transition Schema

Defines the structure for contract state transitions.

Property	Type	Required	Description
commit_hash	string		Git commit hash when transition occurred
environment	string		Environment where transition occurred
file	string		File path that underwent the transition
from	string		Previous state before transition
hash	string		SHA-256 hash of the file content at transition time
metadata	object		Additional transition-specific metadata
reason	string		Human-readable reason for transition

Property	Type	Required	Description
timestamp	string		ISO 8601 timestamp of the transition
to	string		New state after transition
tool	string		Tool that performed the transition
transition	string		Type of transition that occurred
user	string		User who triggered the transition

1.5 Merkle Proof Schema

Defines the structure for Merkle chain validation proofs.

Property	Type	Required	Description
merkle_proof	object		
scope	object		
validation	object		

1.6 WIT Interface Definitions

The following WIT interfaces expose contract validation functionality:

1.7 Hooksmith CLI Interface

Main CLI interface for hook building and management.

```
package hooksmith:cli;

/// Configuration for hook building
record hook-config {
  /// Name of the hook to build
  name: string,
  /// Source directory for the hook
  source-dir: string,
  /// Output directory for built binaries
  output-dir: string,
  /// Whether to include WASM components
  include-wasm: bool,
  /// WASM component paths to include
  wasm-components: list<string>,
}
```

```

/// Result of a hook building operation
record build-result {
  /// Whether the build was successful
  success: bool,
  /// Output path of the built binary
  binary-path: option<string>,
  /// Error message if build failed
  error: option<string>,
  /// Build duration in milliseconds
  duration-ms: u64,
}

/// Hook metadata information
record hook-info {
  /// Hook name
  name: string,
  /// Hook description
  description: string,
  /// Whether the hook is enabled
  enabled: bool,
  /// Hook file path
  path: string,
  /// Hook type (pre-commit, pre-push, etc.)
  hook-type: string,
}

/// Main CLI interface for Hooksmith
interface hooksmith-cli {
  /// Build a hook from source
  build-hook: func(config: hook-config) -> result<build-result, string>;

  /// List all available hooks
  list-hooks: func() -> result<list<hook-info>, string>;

  /// Install hooks into Git repository
  install-hooks: func(hook-names: list<string>) -> result<unit, string>;

  /// Generate Lefthook configuration
  generate-config: func(output-path: string) -> result<unit, string>;

  /// Validate hook configuration
  validate-config: func(config: hook-config) -> result<unit, string>;
}

/// Export the main CLI interface

```

```
export hooksmith-cli;
```

1.8 Hook Builder Interface

Interface for building and managing Git hooks.

```
package hooksmith:hook-builder;

/// Configuration for building a hook
record build-config {
  /// Source path for the hook
  source-path: string,
  /// Output path for the built binary
  output-path: string,
  /// Target triple for compilation (e.g., "x86_64-unknown-linux-gnu")
  target-triple: option<string>,
  /// Optimization level (0-3)
  optimization-level: u8,
  /// Whether to include debug symbols
  debug-symbols: bool,
  /// Additional cargo features to enable
  features: list<string>,
  /// Whether to enable all features
  all-features: bool,
  /// Additional environment variables
  env-vars: list<env-var>,
  /// Whether to run tests after building
  run-tests: bool,
  /// Whether to run clippy checks
  run-clippy: bool,
}

/// Environment variable for build process
record env-var {
  /// Variable name
  name: string,
  /// Variable value
  value: string,
}

/// Result of a hook build operation
record build-result {
  /// Whether the build was successful
  success: bool,
  /// Path to the built binary
  binary-path: option<string>,
```

```

    /// Build artifacts (additional files)
    artifacts: list<string>,
    /// Build logs
    build-logs: string,
    /// Error message if build failed
    error: option<string>,
    /// Build duration in milliseconds
    duration-ms: u64,
    /// Binary size in bytes
    binary-size: option<u64>,
    /// Build metadata
    metadata: build-metadata,
}

/// Metadata about the build
record build-metadata {
    /// Rust version used
    rust-version: string,
    /// Cargo version used
    cargo-version: string,
    /// Target triple used
    target-triple: string,
    /// Build timestamp
    timestamp: string,
    /// Build hash for caching
    build-hash: string,
}

/// Configuration for source validation
record validation-config {
    /// Source path to validate
    source-path: string,
    /// Validation rules to apply
    rules: list<validation-rule>,
    /// Whether to check for common issues
    check-common-issues: bool,
    /// Whether to validate dependencies
    validate-dependencies: bool,
}

/// Validation rule to apply
record validation-rule {
    /// Rule name
    name: string,
    /// Rule description
    description: string,

```

```

    /// Whether the rule is enabled
    enabled: bool,
    /// Rule severity
    severity: rule-severity,
}

/// Severity of a validation rule
enum rule-severity {
    /// Information only
    info,
    /// Warning
    warning,
    /// Error
    error,
}

/// Result of source validation
record validation-result {
    /// Whether validation was successful
    success: bool,
    /// Validation errors
    errors: list<validation-error>,
    /// Validation warnings
    warnings: list<validation-warning>,
    /// Validation information
    info: list<validation-info>,
    /// Validation duration in milliseconds
    duration-ms: u64,
}

/// Validation error
record validation-error {
    /// Error message
    message: string,
    /// Error location (file:line:column)
    location: option<string>,
    /// Error code
    code: option<string>,
    /// Suggested fix
    suggestion: option<string>,
}

/// Validation warning
record validation-warning {
    /// Warning message
    message: string,

```

```

    /// Warning location (file:line:column)
    location: option<string>,
    /// Warning code
    code: option<string>,
    /// Suggested fix
    suggestion: option<string>,
}

/// Validation information
record validation-info {
    /// Information message
    message: string,
    /// Information location (file:line:column)
    location: option<string>,
    /// Information code
    code: option<string>,
}

/// Configuration for binary optimization
record optimization-config {
    /// Binary path to optimize
    binary-path: string,
    /// Optimization level (0-3)
    level: u8,
    /// Whether to strip debug symbols
    strip-debug: bool,
    /// Whether to strip symbols
    strip-symbols: bool,
    /// Whether to compress the binary
    compress: bool,
    /// Target file size (in bytes)
    target-size: option<u64>,
}

/// Result of binary optimization
record optimization-result {
    /// Whether optimization was successful
    success: bool,
    /// Original binary size
    original-size: u64,
    /// Optimized binary size
    optimized-size: u64,
    /// Size reduction percentage
    reduction-percentage: f64,
    /// Optimization techniques applied
    techniques-applied: list<string>,

```



```

    /// Error message if optimization failed
    error: option<string>,
    /// Optimization duration in milliseconds
    duration-ms: u64,
}

/// Hook builder interface
interface hook-builder {
    /// Build a hook from source
    build-hook: func(config: build-config) -> result<build-result, string>;

    /// Validate hook source code
    validate-source: func(config: validation-config) -> result<validation-result, string>;

    /// Optimize a built binary
    optimize-binary: func(config: optimization-config) -> result<optimization-result, string>;

    /// Get build information
    get-build-info: func() -> result<build-metadata, string>;

    /// Clean build artifacts
    clean-build: func(build-path: string) -> result<unit, string>;

    /// Get available targets
    get-available-targets: func() -> result<list<string>, string>;

    /// Check if target is supported
    is-target-supported: func(target: string) -> result<bool, string>;
}

/// Export the hook builder interface
export hook-builder;

```

1.9 Validation Interface

Interface for contract validation and state machine management.

```

package hooksmith:validation;

/// Configuration for validation
record validation-config {
    /// Type of validation to perform
    validation-type: validation-type,
    /// Data to validate
    data: string,
    /// Schema to validate against (if applicable)

```

```

schema: option<string>,
/// Validation rules to apply
rules: list<validation-rule>,
/// Whether to enable strict mode
strict-mode: bool,
/// Whether to include warnings
include-warnings: bool,
/// Whether to include suggestions
include-suggestions: bool,
/// Maximum number of errors to report
max-errors: option<u32>,
/// Maximum number of warnings to report
max-warnings: option<u32>,
}

/// Type of validation to perform
enum validation-type {
/// Validate Lefthook configuration
lefthook-config,
/// Validate hook configuration
hook-config,
/// Validate WIT interface
wit-interface,
/// Validate build configuration
build-config,
/// Validate JSON data
json-data,
/// Validate YAML data
yaml-data,
/// Validate TOML data
toml-data,
/// Validate Rust code
rust-code,
/// Validate shell script
shell-script,
/// Validate file path
file-path,
/// Validate URL
url,
/// Validate email
email,
/// Validate version string
version,
/// Validate semver
semver,
/// Validate custom format

```

```

    custom,
}

/// Validation rule to apply
record validation-rule {
    /// Rule name
    name: string,
    /// Rule description
    description: string,
    /// Whether the rule is enabled
    enabled: bool,
    /// Rule severity
    severity: rule-severity,
    /// Rule pattern (regex or glob)
    pattern: option<string>,
    /// Rule parameters
    parameters: list<rule-parameter>,
    /// Rule dependencies
    dependencies: list<string>,
}

/// Severity of a validation rule
enum rule-severity {
    /// Information only
    info,
    /// Warning
    warning,
    /// Error
    error,
    /// Critical error
    critical,
}

/// Parameter for a validation rule
record rule-parameter {
    /// Parameter name
    name: string,
    /// Parameter value
    value: string,
    /// Parameter type
    parameter-type: parameter-type,
    /// Whether the parameter is required
    required: bool,
}

/// Type of rule parameter

```

```

enum parameter-type {
    /// String parameter
    string,
    /// Number parameter
    number,
    /// Boolean parameter
    boolean,
    /// Array parameter
    array,
    /// Object parameter
    object,
    /// Regex parameter
    regex,
    /// Glob parameter
    glob,
    /// Path parameter
    path,
    /// URL parameter
    url,
    /// Email parameter
    email,
    /// Version parameter
    version,
}

/// Result of validation
record validation-result {
    /// Whether validation was successful
    success: bool,
    /// Validation errors
    errors: list<validation-error>,
    /// Validation warnings
    warnings: list<validation-warning>,
    /// Validation information
    info: list<validation-info>,
    /// Validation duration in milliseconds
    duration-ms: u64,
    /// Validation statistics
    statistics: validation-statistics,
    /// Validation metadata
    metadata: validation-metadata,
}

/// Validation error
record validation-error {
    /// Error message

```

```

message: string,
/// Error location (file:line:column or path)
location: option<string>,
/// Error code
code: option<string>,
/// Suggested fix
suggestion: option<string>,
/// Error context
context: option<string>,
/// Error severity
severity: rule-severity,
/// Error rule
rule: option<string>,
}

/// Validation warning
record validation-warning {
  /// Warning message
  message: string,
  /// Warning location (file:line:column or path)
  location: option<string>,
  /// Warning code
  code: option<string>,
  /// Suggested fix
  suggestion: option<string>,
  /// Warning context
  context: option<string>,
  /// Warning severity
  severity: rule-severity,
  /// Warning rule
  rule: option<string>,
}

/// Validation information
record validation-info {
  /// Information message
  message: string,
  /// Information location (file:line:column or path)
  location: option<string>,
  /// Information code
  code: option<string>,
  /// Information context
  context: option<string>,
  /// Information rule
  rule: option<string>,
}

```

```

/// Validation statistics
record validation-statistics {
  /// Total number of items validated
  total-items: u32,
  /// Number of valid items
  valid-items: u32,
  /// Number of invalid items
  invalid-items: u32,
  /// Number of errors
  error-count: u32,
  /// Number of warnings
  warning-count: u32,
  /// Number of info messages
  info-count: u32,
  /// Validation coverage percentage
  coverage-percentage: f64,
}

/// Validation metadata
record validation-metadata {
  /// Validation timestamp
  timestamp: string,
  /// Validation duration
  duration: string,
  /// Validation type
  validation-type: validation-type,
  /// Validation schema used
  schema-used: option<string>,
  /// Validation rules applied
  rules-applied: list<string>,
  /// Validation environment
  environment: validation-environment,
}

/// Validation environment information
record validation-environment {
  /// Tool version
  tool-version: string,
  /// Platform information
  platform: string,
  /// Validation engine
  engine: string,
  /// Engine version
  engine-version: string,
  /// Available validators

```

```

    available-validators: list<string>,
}

/// Schema validation configuration
record schema-validation-config {
    /// Schema content
    schema: string,
    /// Data to validate
    data: string,
    /// Schema format
    schema-format: schema-format,
    /// Whether to use strict mode
    strict-mode: bool,
    /// Whether to allow additional properties
    allow-additional-properties: bool,
    /// Whether to allow unknown formats
    allow-unknown-formats: bool,
    /// Custom format validators
    format-validators: list<format-validator>,
}

/// Schema format
enum schema-format {
    /// JSON Schema
    json-schema,
    /// OpenAPI
    openapi,
    /// GraphQL
    graphql,
    /// Protocol Buffers
    protobuf,
    /// XML Schema
    xml-schema,
    /// Custom format
    custom,
}

/// Custom format validator
record format-validator {
    /// Format name
    name: string,
    /// Format pattern (regex)
    pattern: string,
    /// Format description
    description: string,
    /// Format example

```

```

    example: string,
}

/// Result of schema validation
record schema-validation-result {
    /// Whether validation was successful
    success: bool,
    /// Validation errors
    errors: list<schema-validation-error>,
    /// Validation warnings
    warnings: list<schema-validation-warning>,
    /// Validation duration in milliseconds
    duration-ms: u64,
    /// Schema information
    schema-info: schema-info,
}

/// Schema validation error
record schema-validation-error {
    /// Error message
    message: string,
    /// Error path in the data
    path: option<string>,
    /// Error schema path
    schema-path: option<string>,
    /// Error code
    code: option<string>,
    /// Suggested fix
    suggestion: option<string>,
    /// Error details
    details: option<string>,
}

/// Schema validation warning
record schema-validation-warning {
    /// Warning message
    message: string,
    /// Warning path in the data
    path: option<string>,
    /// Warning schema path
    schema-path: option<string>,
    /// Warning code
    code: option<string>,
    /// Suggested fix
    suggestion: option<string>,
    /// Warning details

```



```

    details: option<string>,
}

/// Schema information
record schema-info {
    /// Schema title
    title: option<string>,
    /// Schema description
    description: option<string>,
    /// Schema version
    version: option<string>,
    /// Schema format
    format: schema-format,
    /// Schema properties count
    properties-count: u32,
    /// Schema required properties
    required-properties: list<string>,
    /// Schema examples
    examples: list<string>,
}

/// Validation interface
interface validation {
    /// Validate data according to configuration
    validate: func(config: validation-config) -> result<validation-result, string>;

    /// Validate data against a schema
    validate-schema: func(config: schema-validation-config) -> result<schema-validation-result, string>;

    /// Get available validation types
    get-available-types: func() -> result<list<string>, string>;

    /// Get validation type information
    get-type-info: func(validation-type: validation-type) -> result<validation-type-info, string>;

    /// Get available validation rules
    get-available-rules: func() -> result<list<validation-rule>, string>;

    /// Get rule information
    get-rule-info: func(rule-name: string) -> result<validation-rule, string>;

    /// Test a validation rule
    test-rule: func(rule: validation-rule, data: string) -> result<validation-result, string>;

    /// Get validation statistics
    get-statistics: func() -> result<validation-statistics, string>;
}

```

```

    /// Clear validation cache
    clear-cache: func() -> result<unit, string>;
}

/// Information about a validation type
record validation-type-info {
    /// Type name
    name: string,
    /// Type description
    description: string,
    /// Type category
    category: string,
    /// Supported formats
    supported-formats: list<string>,
    /// Required parameters
    required-parameters: list<string>,
    /// Optional parameters
    optional-parameters: list<string>,
    /// Examples
    examples: list<string>,
    /// Documentation
    documentation: option<string>,
}

/// Export the validation interface
export validation;

```

1.10 Lefthook Generator Interface

Interface for generating Lefthook configurations.

```

package hooksmith:lefthook-generator;

/// Configuration for Lefthook generation
record lefthook-config {
    /// Output path for the configuration file
    output-path: string,
    /// Hook configurations to include
    hooks: list<hook-config>,
    /// Whether to validate against schema
    validate-schema: bool,
    /// Global settings for Lefthook
    global-settings: global-settings,
    /// Git repository information
    git-info: git-info,
}

```

```

    /// Whether to include comments in generated config
    include-comments: bool,
    /// Whether to use strict mode
    strict-mode: bool,
}

/// Configuration for a single hook
record hook-config {
    /// Hook name
    name: string,
    /// Hook type (pre-commit, pre-push, etc.)
    hook-type: string,
    /// Command to execute
    command: string,
    /// Whether the hook is enabled
    enabled: bool,
    /// Hook description
    description: option<string>,
    /// Hook tags
    tags: list<string>,
    /// Hook stage (pre, post)
    stage: option<string>,
    /// Hook priority (lower numbers = higher priority)
    priority: option<u32>,
    /// Hook timeout in seconds
    timeout: option<u32>,
    /// Hook working directory
    working-dir: option<string>,
    /// Hook environment variables
    env: list<env-var>,
    /// Hook dependencies
    depends-on: list<string>,
    /// Hook skip conditions
    skip: list<skip-condition>,
}

/// Environment variable for hook execution
record env-var {
    /// Variable name
    name: string,
    /// Variable value
    value: string,
}

/// Skip condition for hook execution
record skip-condition {

```

```

    /// Condition type
    condition-type: skip-condition-type,
    /// Condition value
    value: string,
    /// Condition description
    description: option<string>,
}

/// Type of skip condition
enum skip-condition-type {
    /// Skip if file matches pattern
    file-pattern,
    /// Skip if branch matches pattern
    branch-pattern,
    /// Skip if commit message matches pattern
    commit-message,
    /// Skip if merge commit
    merge-commit,
    /// Skip if no files changed
    no-files-changed,
    /// Skip if specific files changed
    files-changed,
    /// Skip if environment variable is set
    env-var,
    /// Skip if command returns true
    command,
}

/// Global settings for Lefthook
record global-settings {
    /// Lefthook version
    version: option<string>,
    /// Git hooks directory
    git-hooks-dir: option<string>,
    /// Whether to skip output
    skip-output: option<bool>,
    /// Whether to use colors
    colors: option<bool>,
    /// Whether to use parallel execution
    parallel: option<bool>,
    /// Maximum parallel tasks
    max-parallel: option<u32>,
    /// Whether to stop on first failure
    stop-on-failure: option<bool>,
    /// Whether to run only staged files
    only-staged: option<bool>,
}

```

```

    /// Whether to run only changed files
    only-changed: option<bool>,
    /// Whether to run only specific files
    only-files: option<list<string>>,
    /// Whether to run only specific directories
    only-dirs: option<list<string>>,
    /// Whether to run only specific extensions
    only-extensions: option<list<string>>,
    /// Whether to run only specific patterns
    only-patterns: option<list<string>>,
    /// Whether to run only specific branches
    only-branches: option<list<string>>,
    /// Whether to run only specific tags
    only-tags: option<list<string>>,
    /// Whether to run only specific commits
    only-commits: option<list<string>>,
    /// Whether to run only specific authors
    only-authors: option<list<string>>,
    /// Whether to run only specific emails
    only-emails: option<list<string>>,
    /// Whether to run only specific messages
    only-messages: option<list<string>>,
    /// Whether to run only specific files
    only-files-changed: option<list<string>>,
    /// Whether to run only specific directories
    only-dirs-changed: option<list<string>>,
    /// Whether to run only specific extensions
    only-extensions-changed: option<list<string>>,
    /// Whether to run only specific patterns
    only-patterns-changed: option<list<string>>,
}

/// Git repository information
record git-info {
    /// Repository URL
    repo-url: option<string>,
    /// Repository name
    repo-name: option<string>,
    /// Default branch
    default-branch: option<string>,
    /// Current branch
    current-branch: option<string>,
    /// Repository root
    repo-root: option<string>,
    /// Git version
    git-version: option<string>,

```

```

    /// Whether repository is bare
    is-bare: option<bool>,
    /// Whether repository is shallow
    is-shallow: option<bool>,
}

/// Result of Lefthook configuration generation
record lefthook-result {
    /// Whether generation was successful
    success: bool,
    /// Generated configuration content
    config-content: option<string>,
    /// Output file path
    output-path: option<string>,
    /// Generated hooks count
    hooks-count: u32,
    /// Validation errors
    validation-errors: list<string>,
    /// Validation warnings
    validation-warnings: list<string>,
    /// Error message if generation failed
    error: option<string>,
    /// Generation duration in milliseconds
    duration-ms: u64,
}

/// Configuration for validation
record validation-config {
    /// Configuration content to validate
    config-content: string,
    /// Schema to validate against
    schema: option<string>,
    /// Whether to validate syntax
    validate-syntax: bool,
    /// Whether to validate schema
    validate-schema: bool,
    /// Whether to validate hooks
    validate-hooks: bool,
    /// Whether to validate commands
    validate-commands: bool,
}

/// Result of configuration validation
record validation-result {
    /// Whether validation was successful
    success: bool,

```

```

    /// Validation errors
    errors: list<validation-error>,
    /// Validation warnings
    warnings: list<validation-warning>,
    /// Validation information
    info: list<validation-info>,
    /// Validation duration in milliseconds
    duration-ms: u64,
}

/// Validation error
record validation-error {
    /// Error message
    message: string,
    /// Error location (line:column)
    location: option<string>,
    /// Error code
    code: option<string>,
    /// Suggested fix
    suggestion: option<string>,
}

/// Validation warning
record validation-warning {
    /// Warning message
    message: string,
    /// Warning location (line:column)
    location: option<string>,
    /// Warning code
    code: option<string>,
    /// Suggested fix
    suggestion: option<string>,
}

/// Validation information
record validation-info {
    /// Information message
    message: string,
    /// Information location (line:column)
    location: option<string>,
    /// Information code
    code: option<string>,
}

/// Lefthook generator interface
interface lefthook-generator {

```

```

/// Generate Lefthook configuration
generate-config: func(config: lefthook-config) -> result<lefthook-result, string>;

/// Validate Lefthook configuration
validate-config: func(config: validation-config) -> result<validation-result, string>;

/// Get default configuration template
get-default-template: func() -> result<string, string>;

/// Get configuration schema
get-config-schema: func() -> result<string, string>;

/// Parse existing configuration
parse-config: func(config-path: string) -> result<lefthook-config, string>;

/// Merge configurations
merge-configs: func(configs: list<lefthook-config>) -> result<lefthook-config, string>;

/// Get available hook types
get-available-hook-types: func() -> result<list<string>, string>;

/// Get hook type information
get-hook-type-info: func(hook-type: string) -> result<hook-type-info, string>;
}

/// Information about a hook type
record hook-type-info {
  /// Hook type name
  name: string,
  /// Hook type description
  description: string,
  /// Hook type stage (pre, post)
  stage: string,
  /// Hook type triggers
  triggers: list<string>,
  /// Hook type examples
  examples: list<string>,
  /// Hook type documentation
  documentation: option<string>,
}

/// Export the lefthook generator interface
export lefthook-generator;

```


1.11 Integration with WIT & JSON Schema

This implementation demonstrates how JSON Schema and WIT can work together:

1. **JSON Schema Defines the Contract State Machine** - Schemas enforce structure and validation rules
2. **WIT Interface Exposes Contract Validation** - WASM components can validate and transition states
3. **WASM Component Implements Logic** - Components can return schemas, validate states, and apply transitions
4. **Rust Host Uses Both** - Combines schemars and wit-bindgen for type-safe validation

1.12 Benefits

- **Schema as Single Source of Truth** – JSON Schema defines the valid state machine
- **Language-agnostic Validation** – Any host that supports WIT/WASM can validate contracts
- **Deterministic Contract Proofs** – The same logic works inside and outside Git hooks
- **Portable Across Hosts** – Works with Rust, Node.js, Deno, or any WASM runtime